

Reviewer Pack — Real Rank Lower Bound for ACC^0 Circuit Size

Entry 2af1f3860e05 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-29 07:53:03 UTC

Real Rank Lower Bound for ACC^0 Circuit Size

Entry ID: 2af1f3860e05 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-29 07:53:03 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): ACC^0 Circuit Complexity

Statement:

The real rank of the communication matrix of a Boolean function $f: \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}$ is $\Omega(\log n)$ if and only if f cannot be computed by a polynomial-size ACC^0 circuit.

Rationale (proposer’s reasoning):

Williams’ approach leverages algebraic rank properties of matrices to establish lower bounds. By analyzing the real rank of the communication matrix, we can derive constraints on the circuit size, extending tropical rank analysis to real semialgebraic geometry. High real rank implies structural complexity that resists ACC^0 computation.

Taxonomy category: ACC_LB_via_WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c3488a6fdaa3bb42

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.3s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    n = 64
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_matrix(f):
        M = [[0] * (2**n) for _ in range(2**n)]
        for i in range(2**n):
            for j in range(2**n):
                a = bin(i)[2:].zfill(n)
                b = bin(j)[2:].zfill(n)
                M[i][j] = f[int(a + b, 2)]
        return M

    def gaussian_elimination(M):
        m, n = len(M), len(M[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(M[j][i]) > abs(M[max_row][i]):
                    max_row = j
            M[i], M[max_row] = M[max_row], M[i]
            if M[i][i] == 0:
                return None
            for j in range(n-1, i-1, -1):
                M[i][j] /= M[i][i]
            for k in range(m):
                if k != i and M[k][i] != 0:
                    factor = M[k][i]
                    for j in range(i, n):
                        M[k][j] -= factor * M[i][j]
        return M

    def real_rank(M):
        M = gaussian_elimination(M)
        if M is None:
            return 0
        rank = sum(1 for row in M if any(row))
        return rank
```

```

f = generate_boolean_function(n)
M = communication_matrix(f)
rank = real_rank(M)

conjecture_holds = rank >= math.log2(n)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "real_rank",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 6)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction=1.0")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results; insufficient data to evaluate support fraction or counterexamples. | next: Increase timeout duration and re-run test with extended computation time

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	94071
2	preregistration	ollama_remote	qwen3:8b	0	0	24032
3	novelty	ollama_remote	qwen3:8b	0	0	20582
4	novelty	ollama_remote	qwen3:8b	0	0	14498
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12540
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8726
7	judge	ollama_remote	qwen3:8b	0	0	36680

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 211129 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/2af1f3860e05.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/2af1f3860e05.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/2af1f3860e05.tar.gz (if generated)